

School: SET		Batch: 2024-2028																					
Program: B Tech.		Current Academic Year: 2026-27																					
Branch: CSE		Semester: V																					
1	Course Code	CSE357																					
2	Course Title	Software Engineering and Testing Methodologies																					
3	Credits	2																					
4	Contact Hours (L-T-P-SL)	2-0-0-2																					
	Course Status	Core																					
5	Course Objective	The objective of this course is to provide 1. Fundamental knowledge of software engineering 2. To make students aware of best software engineering practices 3. Inculcate ability in students to work as an effective member or leader of software engineering teams 4. To help students to develop skills that will enable them to construct software of high quality																					
6	Course Outcomes	The student would be able to: CO1: Illustrate and compare an effective software engineering process, based on knowledge of widely used development lifecycle model CO2: Apply effective requirement elicitation techniques to develop SRS for a project. CO3: Construct design documents with the help of designing tools CO4: Analyze testing strategies for a software system CO5: Develop and deliver quality software as an individual or as part of a multidisciplinary team. CO6: Adapt techniques and tools necessary for software engineering practices.																					
7	Course Description	This course provides knowledge of software engineering. It introduces concepts such as software processes and agile methods and essential software development activities, from initial specification to system maintenance. Formalisms and tools to assist in software development are also presented, including common design patterns and UML notation. Course focuses on all levels of testing.																					
8	Teaching and Learning Scheme (in Hours per Semester)	<table><tr><td colspan="2">Classroom Instructions (CI)</td><td>Lab Instructions (LI)</td><td>Teamwork (TW) and Self Learning (SL) (TW+SL)</td><td>Total number of Hours</td><td>Total Credits TH/30</td></tr><tr><td>L</td><td>T</td><td>P</td><td>SL</td><td>TH</td><td></td></tr><tr><td>30</td><td>0</td><td>0</td><td>30</td><td>60</td><td>2</td></tr></table>				Classroom Instructions (CI)		Lab Instructions (LI)	Teamwork (TW) and Self Learning (SL) (TW+SL)	Total number of Hours	Total Credits TH/30	L	T	P	SL	TH		30	0	0	30	60	2
Classroom Instructions (CI)		Lab Instructions (LI)	Teamwork (TW) and Self Learning (SL) (TW+SL)	Total number of Hours	Total Credits TH/30																		
L	T	P	SL	TH																			
30	0	0	30	60	2																		
9	Outline syllabus				CO Mapping																		
	Unit 1	Introduction to Software Engineering and Process Models																					

	A	Significance challenges and Software Myths in software engineering, Software Components, Software Characteristics, Software Crisis, software applications	CO1
	B	Software Development Methodologies: Waterfall model, prototyping model, Incremental model, Spiral model, V model, component based, RAD model	CO1
	C	Agility, Agile Process models: Extreme Programming (XP), Adaptive Software Development (ASD), Scrum	CO1
	Unit 2	Software Requirement Engineering	
	A	Requirement Elicitation: Interviews, Brainstorming Sessions, Quality Function Deployment.	CO2
	B	Functional & Non-Functional Requirements, Known Requirements, Unknown Requirements, Undreamt Requirement.	CO2
	C	Requirement Documentation: Characteristics of SRS, Document SRS according to IEEE standards.	CO2
	Unit 3	Software Design	
	A	Design Concepts, Design Strategies: Function Oriented Design, Object Oriented Design, Top-Down and Bottom-Up Design	CO3, CO6
	B	Effective modular design: Functional independence, Cohesion, Coupling, Design documentation	CO3, CO6
	C	UML Diagrams and Tools: Introduction to UML Diagrams, Use Case, Object and Class, Interaction diagram: Sequence & Collaboration, Introduction to Rational Rose tool	CO3, CO6
	Unit 4	Software Implementation and Testing	
	A	Fundamentals of testing: Objectives, principles, myths and facts, Error, Mistake, Bug, Fault and Failure, limitations of testing, AWS Testing Tools and Services, Automated AWS testing tools, AWS Third-party QA tools, Do-it-yourself QA in AWS	CO4
	B	Levels of testing: Unit Testing, Integration Testing, System Testing, Acceptance Testing: Alpha & Beta Testing, Integration techniques, Penetration Testing in AWS, Distributed Load Testing on AWS,	CO4, CO6
	C	White Box Testing, Black Box Testing, Verification and Validation, Test case designing, Guidelines for Coding, Debugging	CO4, CO6
	Unit 5	Maintenance & Quality Management	
	A	Introduction to Maintenance, Need for Maintenance, Categories of Maintenance: Preventive, Corrective and Perfective Maintenance, Cost of Maintenance	CO5, CO6
	B	Quality Concepts: Quality, Quality Control, Cost of Quality, Software Quality Assurance, SQA Plan, Software	CO5, CO6

		Reliability: Measures of Reliability and Availability, Software Safety, Cloud QA Testing on AWS			
	C	Statistical Software Quality Assurance: Six Sigma, The ISO 9000 Quality Standards, Capability Maturity Model			CO5, CO6
10	List of activities for Self-Learning (SL: 30h)	S. No.	Name of the Activity	No of Hours	Evaluation Criteria
		1	Assignment writing on estimation, scheduling, earned value, and risk analysis	5 assignments of 3h each. Total = 15h	Based on the assignment submitted.
		2	Technical Video based learning related to software project management	Duration of video = 5h; Report preparation & Presentation = 10h; Total = 15h	Report/presentation based on the video learning outcomes.
11	Mode of examination	Theory			
	Weightage Distribution	CA	MSE	ESE	
		25%	15%	60%	
12	Textbook/s*	1. Pressman R S, Software Engineering: A Practitioners Approach, McGraw Hill.			
	Other References	1. Datta S, Software Engineering: Concepts and Applications, Oxford University Press, 2010. 2. K.K. Aggrawal and Yogesh Singh, "Software Engineering", New Age International Publication 3. Sommerville, Ian. "Software Engineering", Pearson (Latest Ed). 4. <u>Distributed Load Testing on AWS - Implementation Guide (amazon.com)</u> 5. <u>Automate the testing of your software applications at scale using the Distributed Load Testing on AWS solution - Distributed Load Testing on AWS (amazon.com)</u> 6. <u>AWS For Testers And QA AWS Testing Tools And Services (k2lacademy.com)</u> 7. <u>Public DDoS Testing Policy - Amazon Web Services (AWS)</u> 8. <u>Penetration Testing - Amazon Web Services (AWS)</u>			

CO and PO Mapping

S. No.	Course Outcomes (COs)	High and Moderately Mapped POs and PSOs
1.	CO1: Illustrate and compare an effective software engineering process, based on knowledge of widely used development lifecycle model	PO1, PO2, PSO1
2.	CO2: Apply effective requirement elicitation techniques to develop SRS for a project	PO1, PO2, PO3, PSO1
3.	CO3: Construct design documents with the help of designing tools	PO1, PO2, PO3, PO5, PSO1
4.	CO4: Analyze testing strategies for a software system	PO2, PO3, PO4, PO5, PSO1, PSO2

5.	CO5: Develop and deliver quality software as an individual or as part of a multidisciplinary team	PO2, PO3, PO5, PO9, PO10, PSO1, PSO2
6.	CO6: Adapt techniques and tools necessary for software engineering practices	PO1, PO2, PO3, PO5, PO11, PSO1, PSO2

PO and PSO mapping with level of strength for Course Name Software Engineering and Testing Methodologies (Course Code CSE357)

Course Code	COs \ POs/PSOs	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO10	PO11	PSO 1	PSO 2	PSO3
CSE357	CO1	3	2	1	–	1	–	–	–	–	–	1	2	1	–
	CO2	2	3	2	1	1	–	–	–	1	2	–	3	1	–
	CO3	2	2	3	1	3	–	–	–	1	1	–	3	2	1
	CO4	2	3	2	3	2	–	–	–	–	1	–	3	2	–
	CO5	1	2	3	1	2	–	–	1	3	2	1	2	2	2
	CO6	2	2	2	1	3	–	–	–	1	1	2	3	3	1

Average of non-zeros entry in following table (should be auto calculated).

Course Code	PO1	PO2	PO3	PO4	PO5	PO6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
CSE357	2	2.33	2.17	1.4	2	0	0	1	1.5	1.4	1.3	2.67	1.8	1.3

Strength of Correlation: 20.9

1. Addressed to Slight (Low=1) extent
2. Addressed to Moderate (Medium=2) extent
3. Addressed to Substantial (High=3) extent

Course-to-SDG Matrix

Sustainable Development Goals (SDGs) are 17 global targets established by the United Nations in 2015 to end poverty, protect the planet, and ensure prosperity and peace for all by 2030. The course Software Engineering and Testing Methodologies contributes significantly to sustainable technological education and innovation by equipping students with structured software development, requirement engineering, design, and testing skills. It aligns strongly with SDG 4 and SDG 9 by fostering industry-relevant competencies and supporting the development of reliable digital infrastructure. The course also supports SDG 8 through employability-oriented software engineering practices and collaborative development methodologies. Additionally, the focus on software quality and reliability contributes modestly

toward SDG 16 by encouraging the development of trustworthy and dependable software systems.

Mapping uses a 3-point scale:

- Addressed to **Substantial (High=3) extent**: Core content directly contributes
- Addressed to **Moderate (Medium=2) extent**: Supports through skills/activities
- Addressed to **Slight (Low=1) extent**: Indirect awareness/benefit

SDG Goal	SDG Description	Mapping Strength	Justification
SDG 4	Quality Education	3	The course develops foundational and advanced competencies in software engineering, software quality assurance, testing methodologies, and modern development practices, contributing directly to quality technical education and lifelong learning skills.
SDG 8	Decent Work and Economic Growth	2	By emphasizing professional software development practices, teamwork, testing standards, and industry-oriented tools, the course enhances employability and prepares students for productive roles in the software industry.
SDG 9	Industry, Innovation and Infrastructure	3	The course supports innovation and digital infrastructure development through software lifecycle models, requirement engineering, system design, and software quality engineering practices essential for modern technological systems.
SDG 16	Peace, Justice and Strong Institutions	1	Software testing and quality assurance practices promote reliable, secure, and accountable software systems that support transparent and dependable digital services and institutions.

Total Mapped SDGs: 4 out of 17.